

Natural Language Processing

06. Neural Networks

Juan José Alegría

April 22, 2025

Table of contents

1. Linear Models Recap
2. Neural Networks
3. Neural Network Training
4. The Computation Graph Abstraction
5. More on neural networks

Linear Models Recap

Linear models

- A linear model is defined by:

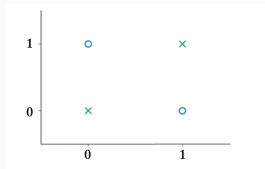
$$f(\vec{x}) = \vec{x} \cdot \vec{W} + \vec{b}$$

$$\vec{x} \in \mathcal{R}^{d_{in}} \quad \vec{W} \in \mathcal{R}^{d_{in} \times d_{out}} \quad \vec{b} \in \mathcal{R}^{d_{out}}$$

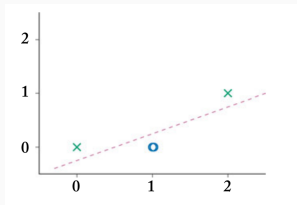
- For binary classification, we can use:
 - $\hat{y} = \text{sign}(f(\vec{x}))$, if we are only interested on the predicted class, or
 - the sigmoid function, $\hat{y} = \sigma(f(\vec{x})) = \frac{1}{1 + e^{-\vec{x} \cdot \vec{W} - b}}$, if we are interested on the probability or confidence on the decision
- For multi-class problems, we can use the softmax function, which transforms the output vector into a probability distribution: $\vec{y}_{[j]} = \frac{e^{(\vec{x} \cdot \vec{W} + \vec{b})_{[j]}}}{\sum_j e^{(\vec{x} \cdot \vec{W} + \vec{b})_{[j]}}}$, and then predict using the *argmax* function.

Linear models

- Linear models cannot represent some functions, such as XOR



- Nevertheless, we can map the values using a non-linear function, and then we can solve the problem



Linear models

- In some cases, we can find the mapping function relatively easily. For instance, for the XOR problem, we can use $\phi(x_1, x_2) = [x_1 \times x_2, x_1 + x_2]$ and then train a classifier to solve $\hat{y} = f(\vec{x}) = \phi(\vec{x}) \cdot \vec{w} + b$
- But we would like to find the ϕ function during training, i.e., to be a trainable function.
- We would like something like

$$\hat{y} = f(\vec{x}) = \phi(\vec{x}) \cdot \vec{w} + b$$

$$\phi(\vec{x}) = g(\vec{x}W' + \vec{b}')$$

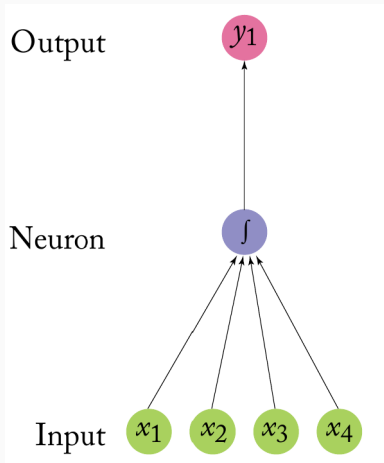
- This is indeed a neural network architecture

Neural Networks

Introduction to Neural Networks

- Very popular machine learning models formed by units called **neurons**.
- A neuron is a computational unit that has scalar inputs and outputs.
- Each input has an associated weight w .
- The neuron multiplies each input by its weight, and then sums them (other functions such as *max* are also possible).
- It applies an activation function g (usually non-linear) to the result, and passes it to its output.
- The nonlinear activation function g has a crucial role in the network's ability to represent complex functions.
- Without the nonlinearity in g , the neural network can only represent linear transformations of the input.

Neuron

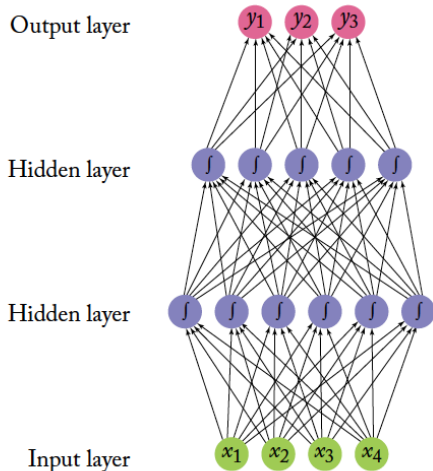


⁰Source:[Goldberg, 2017]

Neural network

- We can connect many neurons, creating a network (hence the name **neural networks**)
- Neurons are arranged in **layers**, reflecting the flow of information
- The bottom layer has no incoming arrows, as is the **input** to the network
- The top-most layer has no outgoing arrows, as is the **output** of the network
- The other layers are called **hidden** layers.
- When each neuron in one layer is connected to all neurons in the next layer, we are talking about a **fully connected layer**. A network comprised of such layers is called a **feedforward neural network** (also called **multilayer perception** (MLP))

Feedforward Network with two Layers



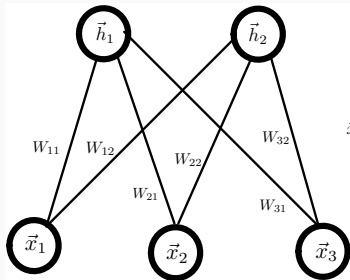
⁰Source:[Goldberg, 2017]

Feedforward Network Neural Networks

- The feedforward network from the picture is a stack of linear models separated by nonlinear functions.
- The values of each row of neurons in the network can be thought of as a vector.
- The input layer is a 4-dimensional vector ($\vec{x}$), and the layer above it is a 6-dimensional vector ($\vec{h}^1$).
- The fully connected layer can be thought of as a linear transformation from 4 dimensions to 6 dimensions.
- A fully connected layer implements a vector-matrix multiplication, $\vec{h} = \vec{x}W + \vec{b}$.
- The weight of the connection from the i -th neuron in the input row to the j -th neuron in the output row is $W_{[i,j]}$.
- The values of $\vec{h}$ are transformed by a nonlinear function g that is applied to each value before being passed on as input to the next layer.

⁰Vectors are assumed to be row vectors and superscript indices correspond to network layers.

Fully connected layers as vector-matrix multiplications



$$\vec{x} = [\vec{x}_1, \vec{x}_2, \vec{x}_3] \quad W = \begin{pmatrix} W_{1,1} & W_{1,2} \\ W_{2,1} & W_{2,2} \\ W_{3,1} & W_{3,2} \end{pmatrix}$$

$$\vec{h} = \vec{x}W$$

$$\vec{x}W = [\vec{x}_1 * W_{11} + \vec{x}_2 * W_{21} + \vec{x}_3 * W_{31}, \vec{x}_1 * W_{12} + \vec{x}_2 * W_{22} + \vec{x}_3 * W_{32}]$$

$$\vec{h} = [\vec{h}_1, \vec{h}_2]$$

Neural Networks as Mathematical Functions

- The Multilayer Perceptron (MLP) from the figure is called MLP2 because it has two hidden layers.
- A simpler model would be MLP1, a multilayer perceptron of one hidden layer:

$$\vec{y} = NN_{MLP1}(\vec{x}) = g(\vec{x}W^1 + \vec{b}^1)W^2 + \vec{b}^2$$
$$\vec{x} \in \mathcal{R}^{d_{in}}, W^1 \in \mathcal{R}^{d_{in} \times d_1}, \vec{b}^1 \in \mathcal{R}^{d_1}, W^2 \in \mathcal{R}^{d_1 \times d_{out}}, \vec{b}^2 \in \mathcal{R}^{d_{out}}, \vec{y} \in \mathcal{R}^{d_{out}}$$

- Here W^1 and $\vec{b}^1$ are a matrix and a bias term for the first linear transformation of the input.
- The function g is a nonlinear function that is applied element-wise (also called a nonlinearity or an activation function).
- W^2 and $\vec{b}^2$ are the matrix and bias term for a second linear transform.
- When describing a neural network, one should specify the dimensions of the layers (d_1), the input (d_{in}), and the output (d_{out}).

Neural Networks as Mathematical Functions

- MLP2 can be written as the following mathematical function:

$$NN_{MLP2}(\vec{x}) = \vec{\hat{y}}$$

$$\vec{h}^1 = \vec{x}W^1 + \vec{b}^1$$

$$\vec{h}^2 = g^1(\vec{h}^1)W^2 + \vec{b}^2$$

$$\vec{y} = g^2(\vec{h}^2)W^3 + \vec{b}^3$$

$$\vec{y} = (g^2(g^1(\vec{x}W^1 + \vec{b}^1)W^2 + \vec{b}^2))W^3 + \vec{b}^3.$$

- The matrices and the bias terms that define the linear transformations are the parameters of the network.
- Like in linear models, it is common to refer to the collection of all parameters as Θ .

Representation Power

- [Hornik et al., 1989] and [Cybenko, 1989] showed that a multilayer perceptron of one hidden layer (MLP1) is a universal approximator.
- MLP1 can approximate all continuous functions on a closed and bounded subset of $\mathcal{R}^n$.
- This may suggest there is no reason to go beyond MLP1 to more complex architectures.
- The result does not say how easy or hard it is to set the parameters based on training data and a specific learning algorithm.
- It also does not guarantee that a training algorithm will find the correct function generating our training data.
- Finally, it does not state how large the hidden layer should be.

⁰For intuition about this theorem, see [Nielsen, 2015], Chapter 4.

Representation Power

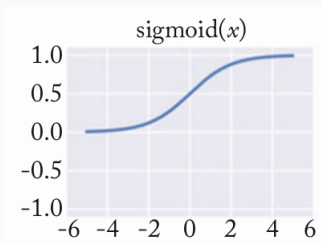
- In practice, we train neural networks using local search methods.
- We also use hidden layers of relatively modest sizes (up to several thousands).
- The universal approximation theorem does not give any guarantees under these conditions.
- However, there is definitely benefit in trying out more complex architectures than MLP1.
- In many cases, however, MLP1 does indeed provide strong results.

Activation Functions

- The nonlinearity g can take many forms.
- There is currently no good theory as to which nonlinearity to apply in which conditions.
- Choosing the correct nonlinearity for a given task is for the most part an empirical question.

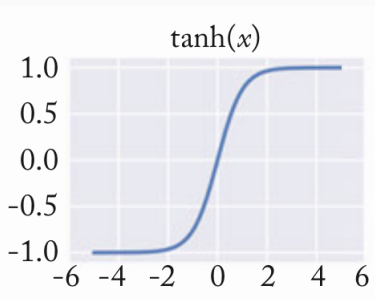
Sigmoid

- The sigmoid activation function $\sigma(x) = \frac{1}{1+e^{-x}}$ is an S-shaped function, transforming each value x into the range $[0, 1]$.
- The sigmoid was the canonical nonlinearity for neural networks since their inception.
- It is currently considered to be deprecated for use in internal layers of neural networks, as the choices listed next prove to work much better empirically.



Hyperbolic tangent (tanh)

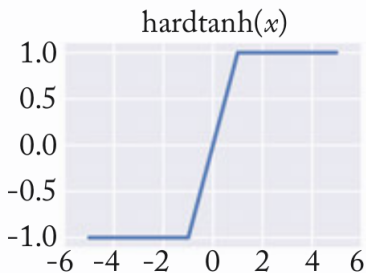
- The hyperbolic tangent $\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1}$ activation function is an S-shaped function, transforming the values x into the range $[-1, 1]$.



Hard tanh

- The hard-tanh activation function is an approximation of the tanh function which is faster to compute and to find derivatives thereof:

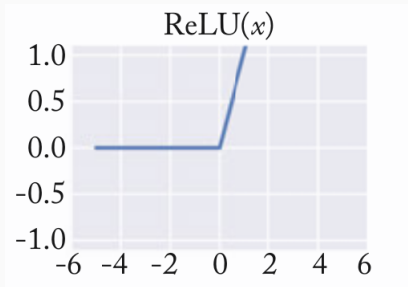
$$\text{hardtanh}(x) = \begin{cases} -1 & x < -1 \\ 1 & x > 1 \\ x & \text{otherwise.} \end{cases}$$



ReLU

- The rectifier activation function [Glorot et al., 2011], also known as the rectified linear unit is a very simple activation function.
- It is easy to work with and was shown many times to produce excellent results.
- The ReLU unit clips each value $x < 0$ at 0.

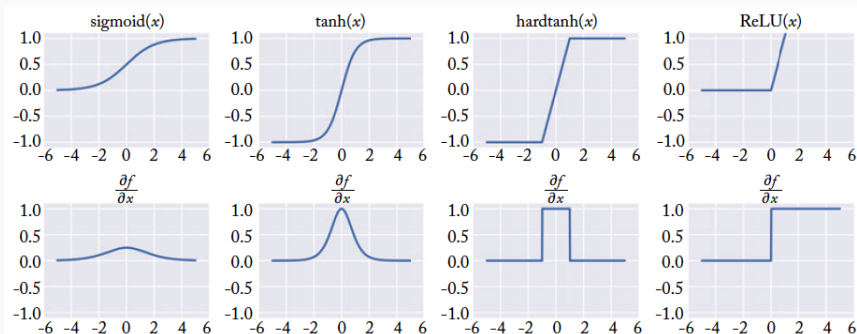
$$\text{ReLU}(x) = \max(0, x)$$



- It performs well for many tasks, especially when combined with the dropout regularization technique (to be explained later).

Activation Functions

- As a rule of thumb, both ReLU and tanh units work well, and significantly outperform the sigmoid.
- You may want to experiment with both tanh and ReLU activations, as each one may perform better in different settings.
- The figure from below shows the shapes of the different activations functions, together with the shapes of their derivatives.



Embedding Layers

- In NLP the input to the neural network contains symbolic categorical features (e.g., words from a closed vocabulary, character n-grams, POS tags).
- In linear models we usually represent the input with sparse vectors e.g., as the sum, average, or the concatenation of one-hot encoded vectors (the sum or the average can produce bag-of words representation).
- In neural networks, it is common to associate each possible feature value (i.e., each word in the vocabulary, each POS tag category) with a d —dimensional dense vector for some d .
- These vectors are then considered parameters of the model, and are trained jointly with the other parameters.
- The mapping from a symbolic feature values such as “word number 1249” to d —dimensional vectors is performed by an embedding layer (also called a lookup layer).

Embedding Layers

- The parameters in a word embedding layer are simply a matrix $E \in \mathcal{R}^{|vocab| \times d}$ where each row corresponds to a different word in the vocabulary.
- The lookup operation is then simply indexing: $v_{1249} = E_{[1249,:]}.$
- If the symbolic feature is encoded as a one-hot vector $\vec{x}$, the lookup operation can be implemented as a vector-matrix multiplication $\vec{x}E$.
- The embedding vectors are combined before being passed on to the next layer.
- Common combination operations are: concatenation, summation, average.
- A word embeddings matrix E can be initialized with pre-trained word vectors trained from unlabeled documents using specific methods based on the distributional hypothesis such as the ones implemented in Word2Vec (to be discussed later in the course).

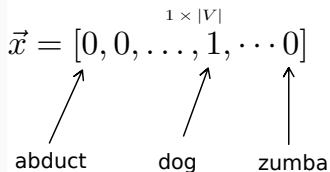
The Embedding Matrix

One-hot-encoded word vector

$$\vec{x} = [0, 0, \dots, 1, \dots 0]$$

$1 \times |V|$

abduct dog zumba

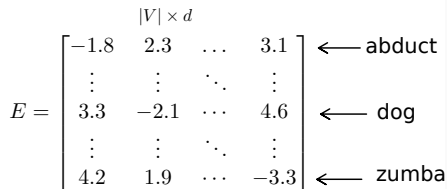


Embedding Matrix

$$E = \begin{bmatrix} -1.8 & 2.3 & \dots & 3.1 \\ \vdots & \vdots & \ddots & \vdots \\ 3.3 & -2.1 & \dots & 4.6 \\ \vdots & \vdots & \ddots & \vdots \\ 4.2 & 1.9 & \dots & -3.3 \end{bmatrix}$$

$|V| \times d$

← abduct
← dog
← zumba



$$\vec{x}E = [3.3, -2.1, \dots, 4.6]$$

Neural Network Training

Neural Network Training

- Suppose that we have defined our architecture. For instance, a feedforward network with two hidden layers. This architecture is parameterized by the set of weights w and biases b of each layer. Let's denote the collection of parameters as Θ , and $\vec{y} = NN(\vec{x}; \Theta)$ the output of the neural network.
- We also have a training set: a collection of input values $\vec{x}_{1:N} = \vec{x}_1, \dots, \vec{x}_N$ and the corresponding desired outputs $\vec{y}_{1:N} = \vec{y}_1, \dots, \vec{y}_N$.
- Let us define a loss function $L(\vec{x}_{1:N}, \vec{y}_{1:N}; \Theta)$ that measures the error of our model on the training set.
- The training goal is to find Θ that minimize the loss:
 $\hat{\Theta} = \operatorname{argmin}_{\Theta} L(\vec{x}_{1:N}, \vec{y}_{1:N}; \Theta)$. Note that $\vec{x}$ and $\vec{y}$ are fixed, they are not parameters of the model. The only values we can control are the weights and biases (i.e., Θ).
- How can we find the optimal values for Θ ?

Neural Network Training

- We know that we can use gradient descent to iteratively update the value of each parameter: $\Theta_i \leftarrow \Theta_i - \eta \frac{\partial L}{\partial \Theta_i}(\hat{y}, y)$ (for all parameters Θ_i and with a learning parameter η).

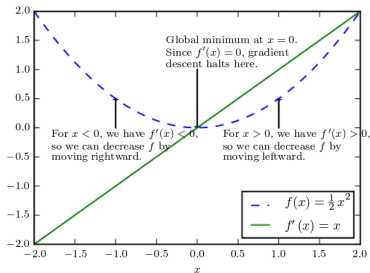


Figure 4.1: Gradient descent. An illustration of how the gradient descent algorithm uses the derivatives of a function to follow the function downhill to a minimum.

- But how can we compute $\frac{\partial L}{\partial \Theta_i}$?

Neural Network Training

- Maybe we can compute the loss directly for each parameter Θ_i , either weight or bias, with the formula $\frac{\partial L}{\partial \Theta_i} \approx \frac{L(\Theta_1, \dots, \Theta_i + \epsilon, \dots, \Theta_M) - L(\Theta_1, \dots, \Theta_i, \dots, \Theta_M)}{\epsilon}$.
- But then, for each Θ_i we would need to compute $L(\Theta_1, \dots, \Theta_i + \epsilon, \dots, \Theta_M)$. If we have a million parameters, we would need to do a million computations.
- We also need to compare with $L(\Theta_1, \dots, \Theta_M)$, so that's one million and one computations. All of that just to perform one update to the parameters of the network, with one training example.
- We would like to compute all the partial derivatives in a faster way.

Backpropagation

- Algorithm to compute the gradient of L with respect to **all** the parameters in a clever way, using the derivative chain rule.
- Plan of attack: do a forward pass with a training example, storing the values at each hidden layer. Then, compute the derivatives iteratively in reverse. Start by calculating the derivatives of the parameters in the last layer, pass the results to the previous layer, then the one before that, and continue until reaching the first layer.
- Goal: compute $\frac{\partial L}{\partial w_{ij}^l}$ and $\frac{\partial L}{\partial b_j^l}$ for each layer l .

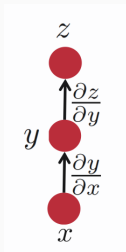
Derivative Chain Rule Recap

- Simple chain rule: let $z = f(y)$, $y = g(x)$,

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x}$$

- Example: $z = e^y$, $y = 2x$

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \times \frac{\partial y}{\partial x} = e^y \times 2 = 2e^{2x}$$



⁰Figure taken from:

<http://cs224d.stanford.edu/lectures/CS224d-Lecture5.pdf>

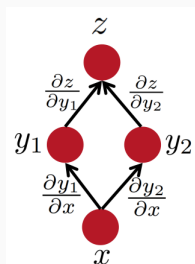
Derivative Chain Rule Recap

- Multiple path chain rule: let $z = f(y_1, y_2)$, $y_1 = g_1(x)$, $y_2 = g_2(x)$

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y_1} \times \frac{\partial y_1}{\partial x} + \frac{\partial z}{\partial y_2} \times \frac{\partial y_2}{\partial x}$$

- Example: $z = e^{y_1 \times y_2}$, $y_1 = 2x$, $y_2 = x^2$

$$\frac{\partial z}{\partial x} = (e^{y_1 \times y_2} \times y_2) \times 2 + (e^{y_1 \times y_2} \times y_1) \times 2x = e^{2x^3} \times 6x^2$$



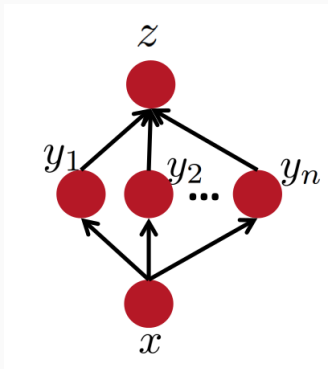
⁰Figure taken from:

<http://cs224d.stanford.edu/lectures/CS224d-Lecture5.pdf>

Derivative Chain Rule Recap

The general version of the multiple path chain rule would be:

$$\frac{\partial z}{\partial x} = \sum_{i=1}^n \frac{\partial z}{\partial y_i} \times \frac{\partial y_i}{\partial x}$$

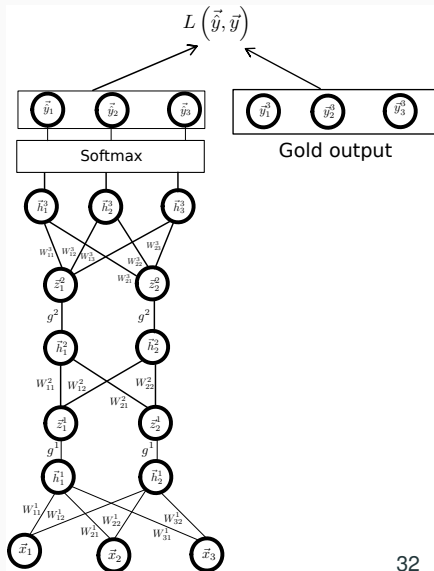


Backpropagation: notation

- M = number of layers in the network
- d_l = dimension of layer l
- $\vec{h}^l = \vec{z}^{l-1} W^l + \vec{b}^l$
- Equivalently,

$$h_j^l = \sum_{i=1}^{d_{l-1}} w_{ij}^l z_i^{l-1} + b_j^l$$
- $\vec{z}^l = g(\vec{h}^l)$. Equivalently,

$$z_j^l = g(h_j^l)$$
- The first layer is just the training sample: $\vec{z}^0 = \vec{x}$.
- The final layer of the network is just $\vec{z}^M = [z_1^M, \dots, z_{d_M}^M]$.
- Just for now, assume that z_j^M depends only on h_j^M . For instance, g on the final layer could be a sigmoid, but not softmax (just for now).
- Finally, a useful value: $\delta_j^l = \frac{\partial L}{\partial h_j^l}$



Backpropagation: equations

We have four equations that describe the backpropagation algorithm:

$$\delta_j^M = \frac{\partial L}{\partial z_j^M} g'(h_j^M) \quad (\text{BP1})$$

$$\delta_j^l = \sum_{k=1}^{d_{l+1}} \delta_k^{l+1} w_{jk}^{l+1} g'(h_j^l) \quad (\text{BP2})$$

$$\frac{\partial L}{\partial b_j^l} = \delta_j^l \quad (\text{BP3})$$

$$\frac{\partial L}{\partial w_{ij}^l} = \delta_j^l z_i^{l-1} \quad (\text{BP4})$$

⁰All the following derivation of backpropagation is adapted from [Nielsen, 2015], Chapter 2.

Backpropagation: equation 1

Goal: find an expression for δ_j^M (i.e., in the last layer).

$$\begin{aligned}\delta_j^M &= \frac{\partial L}{\partial h_j^M} \\ &= \sum_{k=1}^{d_M} \frac{\partial L}{\partial z_k^M} \frac{\partial z_k^M}{\partial h_j^M} \\ &= \frac{\partial L}{\partial z_j^M} \frac{\partial z_j^M}{\partial h_j^M} \quad (\text{since } z_k^M \text{ only depends on } h_k^M, \text{ then } \frac{\partial z_k^M}{\partial h_j^M} = 0 \text{ for } k \neq j) \\ &= \frac{\partial L}{\partial z_j^M} g'(h_j^M) \quad (\text{since } z_j^M = g(h_j^M))\end{aligned}$$

Then:

$$\delta_j^M = \frac{\partial L}{\partial z_j^M} g'(h_j^M) \quad (\text{BP1})$$

Backpropagation: equation 2

Goal: find an expression for δ_j^l (i.e., for a hidden layer)

$$\begin{aligned}\delta_j^l &= \frac{\partial L}{\partial h_j^l} \\ &= \sum_{k=1}^{d_{l+1}} \frac{\partial L}{\partial h_k^{l+1}} \frac{\partial h_k^{l+1}}{\partial h_j^l} \quad (\text{since the output of the layer } l \text{ can affect all the outputs in layer } (l+1)) \\ &= \sum_{k=1}^{d_{l+1}} \delta_k^{l+1} \frac{\partial h_k^{l+1}}{\partial h_j^l} \quad (\text{by definition of } \delta_k^{l+1})\end{aligned}$$

Backpropagation: equation 2

Let's focus on $\frac{\partial h_k^{l+1}}{\partial h_j^l}$.

We know that:

$$\begin{aligned} h_k^{l+1} &= \sum_{i=1}^{d_l} w_{ik}^{l+1} z_i^l + b_k^l \\ &= \sum_{i=1}^{d_l} w_{ik}^{l+1} g(h_i^l) + b_k^l \quad (\text{by definition of } z_i^l) \end{aligned}$$

We note that h_j^l only appears multiplied by w_{jk}^{l+1} . Then, when we compute $\frac{\partial h_k^{l+1}}{\partial h_j^l}$ all the terms with $i \neq j$ will be 0.

$$\frac{\partial h_k^{l+1}}{\partial h_j^l} = w_{jk}^{l+1} g'(h_j^l)$$

Backpropagation: equation 2

Putting everything together:

$$\begin{aligned}\delta_j^l &= \sum_{k=1}^{d_{l+1}} \delta_k^{l+1} \frac{\partial h_k^{l+1}}{\partial h_j^l} \\ &= \sum_{k=1}^{d_{l+1}} \delta_k^{l+1} w_{jk}^{l+1} g'(h_j^l)\end{aligned}\tag{BP2}$$

Alternatively, it can be written in vector form as $\vec{\delta}^l = \vec{\delta}^{l+1} (W^{l+1})^T \odot g'(\vec{h}^l)$, where $\odot$ is the Hadamard or element-wise product.

Backpropagation: equation 3

Goal: find an expression for $\frac{\partial L}{\partial b_j^l}$ (i.e., the derivative of L with respect to the bias of neuron j in layer l)

$$\begin{aligned}\frac{\partial L}{\partial b_j^l} &= \sum_{k=1}^{d_l} \frac{\partial L}{\partial h_k^l} \frac{\partial h_k^l}{\partial b_j^l} \\ &= \frac{\partial L}{\partial h_j^l} \frac{\partial h_j^l}{\partial b_j^l} \quad (\text{since } b_j^l \text{ only affects } h_j^l) \\ &= \delta_j^l \frac{\partial h_j^l}{\partial b_j^l} \quad (\text{by definition of } \delta_j^l) \\ &= \delta_j^l \times 1 \quad (\text{since } h_j^l = \sum w_{ij}^l z_i^{l-1} + b_j^l)\end{aligned}$$

Then:

$$\frac{\partial L}{\partial b_j^l} = \delta_j^l \quad (\text{BP3})$$

Backpropagation: equation 4

Goal: find an expression for $\frac{\partial L}{\partial w_{ij}^l}$ (i.e., the derivative of L with respect to the weight that connects input i -th to neuron j -th in layer l)

$$\begin{aligned}\frac{\partial L}{\partial w_{ij}^l} &= \sum_{k=1}^{d_l} \frac{\partial L}{\partial h_k^l} \frac{\partial h_k^l}{\partial w_{ij}^l} \\ &= \frac{\partial L}{\partial h_j^l} \frac{\partial h_j^l}{\partial w_{ij}^l} \quad (\text{since } w_{ij}^l \text{ only affects } h_j^l) \\ &= \delta_j^l \frac{\partial h_j^l}{\partial w_{ij}^l} \quad (\text{by definition of } \delta_j^l) \\ &= \delta_j^l z_i^{l-1} \quad (\text{since } h_j^l = \sum w_{kj}^l z_k^{l-1} + b_j^l)\end{aligned}$$

Then:

$$\frac{\partial L}{\partial w_{ij}^l} = \delta_j^l z_i^{l-1} \quad (\text{BP4})$$

Backpropagation: algorithm

Let $\vec{x}$ be a training input example, and $\vec{y}$ its corresponding value.

1. **Input:** Set $\vec{z}^0 = \vec{x}$
2. **Feedforward:** For each $l = 1, 2, \dots, M$ compute $\vec{h}^l = \vec{z}^{l-1} W^l + \vec{b}^l$ and $\vec{z}^l = g(\text{vech}^l)$, and store those values.
3. **Output error:** Compute the vector $\nabla \delta^M$, where each component is given by $\delta_j^M = \frac{\partial L}{\partial z_j^M} g'(h_j^M)$.
4. **Backpropagate the error:** for each layer $l = M - 1, M - 2, \dots, 1$, compute $\vec{\delta}^l = (\delta^{l+1} (W^{l+1})^T) \odot g'(\vec{h}^l)$
5. **Update parameters:** update the weights with the rule $w_{ij}^l \leftarrow w_{ij}^l - \frac{\partial L}{\partial w_{ij}^l}$ and the biases with $b_j^l \leftarrow b_j^l - \frac{\partial L}{\partial b_j^l}$. From equations (BP3) and (BP4), we know that $\frac{\partial L}{\partial w_{ij}^l} = \delta_j^l z_j^{l-1}$ and $\frac{\partial L}{\partial b_j^l} = \delta_j^l$

Now we can update the weights with just one forward and one backward pass :)

Backpropagation: softmax?

- When deriving the previous equations, we assume that $z_j^M = g(h_j^M)$; i.e., the activation for the final layer depends only on the value for that neuron.
- That assumption doesn't hold for the softmax function, often used in multi-class classification problems
- $$\text{softmax}(z_j^M) = \frac{e^{z_j^M}}{\sum_k e^{z_k^M}}.$$
- It's easy to derive the equation BP1 in that case, without our previous assumption (see problem *Backpropagation with softmax and the log-likelihood cost*, in [Nielsen, 2015], Chapter 3).
- But it's not that easy to extend this demonstration to other architectures (residual connections, shared weights, etc).
- We need a better abstraction.

The Computation Graph Abstraction

The Computation Graph Abstraction

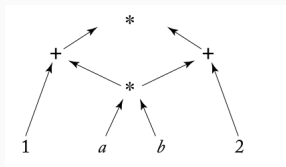
- One can compute the gradients of the various parameters of a network by hand and implement them in code.
- This procedure is cumbersome and error prone.
- For most purposes, it is preferable to use automatic tools for gradient computation [Bengio, 2012].
- A computation graph is a representation of an arbitrary mathematical computation (e.g., a neural network) as a graph.
- This abstraction will allow us computing the gradients from any kind of neural network architecture using the backpropagation algorithm.
- Previous formulation was restricted to feedforward networks.

The Computation Graph Abstraction

- A computation graph is a directed acyclic graph (DAG).
- Nodes correspond to mathematical operations or (bound) variables.
- Edges correspond to the flow of intermediary values between the nodes.
- The graph structure defines the order of the computation in terms of the dependencies between the different components.
- The graph is a DAG and not a tree, as the result of one operation can be the input of several continuations.

The Computation Graph Abstraction

- Consider for example a graph for the computation of $(a * b + 1) * (a * b + 2)$:



- The computation of $a * b$ is shared.
- Since a neural network is essentially a mathematical expression, it can be represented as a computation graph.

The Computation Graph Abstraction

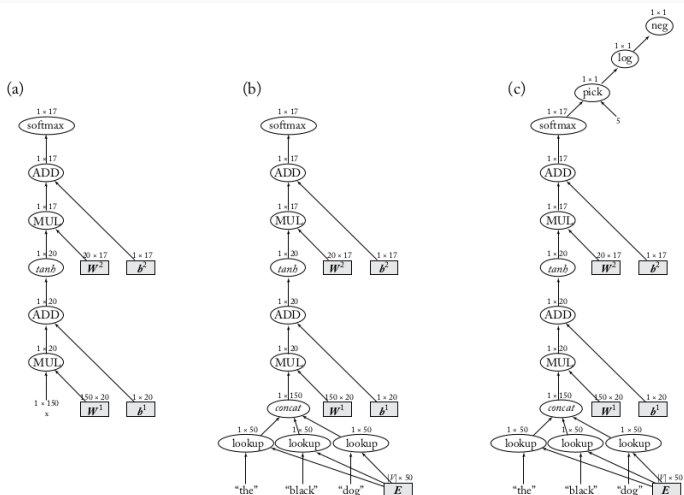


Figure 5.1: (a) Graph with unbound input. (b) Graph with concrete input. (c) Graph with concrete input, expected output, and a final loss node.

The Computation Graph Abstraction

- The figure above shows the computation graph for an MLP with one hidden-layer and a softmax output transformation.
- Oval nodes represent mathematical operations or functions, and shaded rectangle nodes represent parameters (bound variables).
- Network inputs are treated as constants, and drawn without a surrounding node.
- Input and parameter nodes have no incoming arcs, and output nodes have no outgoing arcs.
- The output of each node is a matrix, the dimensionality of which is indicated above the node.

The Computation Graph Abstraction

- This graph is incomplete: without specifying the inputs, we cannot compute an output.
- Figure 5.1b shows a complete graph for an MLP that takes three words as inputs, and predicts the distribution over part-of-speech tags for the third word.
- This graph can be used for prediction, but not for training, as the output is a vector (not a scalar) and the graph does not take into account the correct answer or the loss term.
- Finally, the graph in Figure 5.1c shows the computation graph for a specific training example, in which the inputs are the (embeddings of) the words “the,” “black,” “dog,” and the expected output is “NOUN” (whose index is 5).
- The pick node implements an indexing operation, receiving a vector and an index (in this case, 5) and returning the corresponding entry in the vector.

Forward Computation

- The forward pass computes the outputs of the nodes in the graph.
- Since each node's output depends only on itself and on its incoming edges, it is trivial to compute the outputs of all nodes.
- This is done by traversing the nodes in a topological order and computing the output of each node given the already computed outputs of its predecessors.
- More formally, in a graph of N nodes, we associate each node with an index i according to their topological ordering.
- Let f_i be the function computed by node i (e.g., multiplication, addition , etc.).

Forward Computation

- Let $\pi(i)$ be the parent nodes of node i , and $\pi^{-1}(i) = \{j | i \in \pi(j)\}$ the children nodes of node i (these are the arguments of f_i).
- Denote by $v(i)$ the output of node i , that is, the application of f_i to the output values of its arguments $\pi^{-1}(i)$.
- For variable and input nodes, f_i is a constant function and $\pi^{-1}(i)$ is empty.
- The computation-graph forward pass computes the values $v(i)$ for all $i \in [1, N]$.

Algorithm 1: Computation graph forward pass

for $i = 1$ **to** N **do**

 Let $a_1, \dots, a_m = \pi^{-1}(i)$

$v(i) \leftarrow f_i(v(a_1), \dots, v(a_m))$

end

Backward Computation (Backprop)

- The backward pass begins by designating a node N with scalar (1×1) output as a loss-node, and running forward computation up to that node.
- The backward computation computes the gradients of the parameters with respect to that node's value.
- Denote by $d(i)$ the quantity $\frac{\partial N}{\partial i}$.
- The backpropagation algorithm is used to compute the values $d(i)$ for all nodes i .
- The backward pass fills a table of values $d(1), \dots, d(N)$ as shown in the following algorithm.

Algorithm 2: Computation graph backward pass (backpropagation)

```
 $d(N) \leftarrow 1;$   $// \frac{\partial N}{\partial N} = 1$   
;  
for  $i = N-1$  to  $1$  do  
|  $d(i) = \sum_{j \in \pi(i)} d(j) \cdot \frac{\partial f_j}{\partial i};$   $// \frac{\partial N}{\partial i} = \sum_{j \in \pi(i)} \frac{\partial N}{\partial j} \cdot \frac{\partial j}{\partial i}$   
| ;  
end
```

Backward Computation (Backprop)

- The backpropagation algorithm is essentially following the chain-rule of differentiation.
- The quantity $\frac{\partial f_j}{\partial i}$ is the partial derivative of $f_j(\pi^{-1}(j))$ w.r.t the argument $i \in \pi^{-1}(j)$.
- This value depends on the function f_j and the values $v(a_1), \dots, v(a_m)$ (where $a_1, \dots, a_m = \pi^{-1}(j)$) of its arguments, which were computed in the forward pass.
- Thus, in order to define a new kind of node, one needs to define two methods: one for calculating the forward value $v(i)$ based on the node's inputs, and the another for calculating $\frac{\partial f_j}{\partial i}$ for each $x \in \pi^{-1}(i)$.

Summary of the Computation Graph Abstraction

- Notice that the above formulation of backpropagation is equivalent to one given earlier in the class.
- The computation graph abstraction allows us to:
 1. Easily construct arbitrary networks.
 2. Evaluate their predictions for given inputs (forward pass)
 3. Compute gradients for their parameters with respect to arbitrary scalar losses (backward pass or backpropagation).
- A nice property of the computation graph abstraction is that it allows computing the gradients for arbitrary networks (e.g., networks with skip-connections, shared weights, special loss functions, etc.)

⁰A comprehensive tutorial on the backpropagation algorithm over the computational graph abstraction can be found here:

<https://colah.github.io/posts/2015-08-Backprop/>.

Derivatives of “non-mathematical” functions

- Defining $\frac{\partial f_j}{\partial i}$ for mathematical functions such as $\log$ or $+$ is straightforward.
- It may be challenging to think about the derivative of operations as as $\text{pick}(\vec{x}, 5)$ that selects the fifth element of a vector.
- The answer is to think in terms of the contribution to the computation.
- After picking the i -th element of a vector, only that element participates in the remainder of the computation.
- Thus, the gradient of $\text{pick}(\vec{x}, 5)$ is a vector $\vec{v}$ with the dimensionality of $\vec{x}$ where $\vec{v}_{[5]} = 1$ and $\vec{v}_{[i \neq 5]} = 0$.
- Similarly, for the function $\max(0, x)$ the value of the gradient is 1 for $x > 0$ and 0 otherwise.

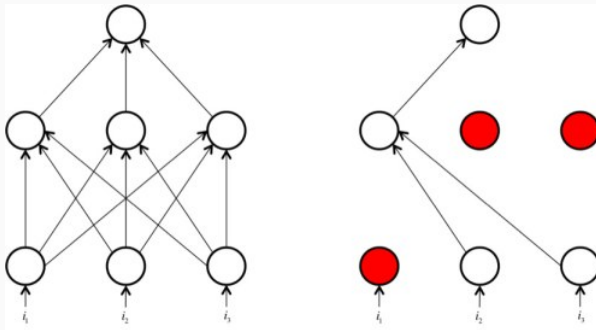
More on neural networks

Regularization and Dropout

- Multi-layer networks can be large and have many parameters, making them especially prone to overfitting.
- Model regularization is just as important in deep neural networks as it is in linear models, and perhaps even more so.
- The regularizers discussed for linear models, namely L_2 , L_1 , and the elastic-net, are also relevant for neural networks.
- Another effective technique for preventing neural networks from overfitting the training data is **dropout training** [Srivastava et al., 2014].

Regularization and Dropout

- The dropout method is designed to prevent the network from learning to rely on specific weights.
- It works by randomly dropping (setting to 0) half of the neurons in the network (or in a specific layer) in each training example in the stochastic-gradient training.



⁰Figure taken from: <https://www.kdnuggets.com/wp-content/uploads/drop-out-in-neural-networks.jpg>

Several software packages implement the computation-graph model. All these packages support all the essential components (node types) for defining a wide range of neural network architectures.

- TensorFlow (<https://www.tensorflow.org/>): an open source software library for numerical computation using data-flow graphs originally developed by the Google Brain Team.
- Keras: High-level neural network API that runs on top of Tensorflow as well as other backends (<https://keras.io/>).
- PyTorch: open source machine learning library for Python, based on Torch, developed by Facebook's artificial-intelligence research group. It supports dynamic graph construction, a different computation graph is created from scratch for each training sample. (<https://pytorch.org/>)

Thanks for your Attention!



Bengio, Y. (2012).

Practical recommendations for gradient-based training of deep architectures.

In *Neural networks: Tricks of the trade*, pages 437–478.
Springer.



Cybenko, G. (1989).

Approximation by superpositions of a sigmoidal function.

Mathematics of control, signals and systems, 2(4):303–314.



Glorot, X., Bordes, A., and Bengio, Y. (2011).

Deep sparse rectifier neural networks.

In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323.



Goldberg, Y. (2017).

Neural network methods for natural language processing.

Synthesis Lectures on Human Language Technologies, 10(1):1–309.



Hornik, K., Stinchcombe, M., and White, H. (1989).

Multilayer feedforward networks are universal approximators.

Neural networks, 2(5):359–366.



Nielsen, M. A. (2015).

Neural Networks and Deep Learning.

Determination Press.



Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014).

Dropout: a simple way to prevent neural networks from overfitting.

The journal of machine learning research,
15(1):1929–1958.